IAM 511: An Introduction to NP-Completeness

Efe Izbudak

November 17, 2025

Contents

1 Introduction **1**

2 A Rigorous Model of Computation **1**

3 Complexity Classes and Reductions **4**
3.1 The Class P 4
3.2 The Class NP 5
3.3 Reducibility and NP-Completeness 6
3.4 The Class coNP 7

4 The SAT and 3SAT Problems **7**

5 The Cook-Levin Theorem **8**

6 Proving NP-complete in Practice **11**
6.1 Example 1: 3SAT to CLIQUE 12
6.2 Example 2: 3SAT to INDSET 13

1 Introduction

These notes provide a rigorous foundation for the theory of NP-completeness, oriented towards graduate students in applied mathematics. The primary goals of this document are to:

1. Formally define a model of computation (the Turing Machine) sufficient for rigorous complexity analysis.
2. Use this model to formally define the complexity classes P, NP, and coNP, and the concept of a polynomial-time reduction.
3. Present a detailed, rigorous proof of the Cook-Levin Theorem (SAT is NP-complete), which forms the bedrock of NP-completeness theory.
4. Demonstrate the practical application of this theorem by showing how to prove other problems (e.g., 3SAT, CLIQUE, INDSET) are NP-complete via reduction.

These notes are intended to be a comprehensive supplement to the lecture, providing the complete formal details that may be covered at a higher level in class.

2 A Rigorous Model of Computation

To discuss complexity, we must first formalize what we mean by an "algorithm." The standard model is the Turing Machine. A problem will be equated with a *language* $L \subseteq \Sigma^*$, which is the set of all input strings for which the answer is "yes."

Informally, a Turing Machine is a mathematical model of a "human computer" following a fixed set of rules. It has a "scratch pad" (the tapes) and a "register" to hold its current state (finite control). Its behavior at any step is determined entirely by its current state and the symbols it is currently reading from its tapes.

A k-tape Turing Machine, as illustrated in Figure 1, typically has:

- An **Input Tape**, which is read-only.
- One or more **Work Tapes**, which are read/write.
- An **Output Tape**, which is (conceptually) write-only.
- A **State Register** (finite control) that holds the current state $q_i \in Q$.

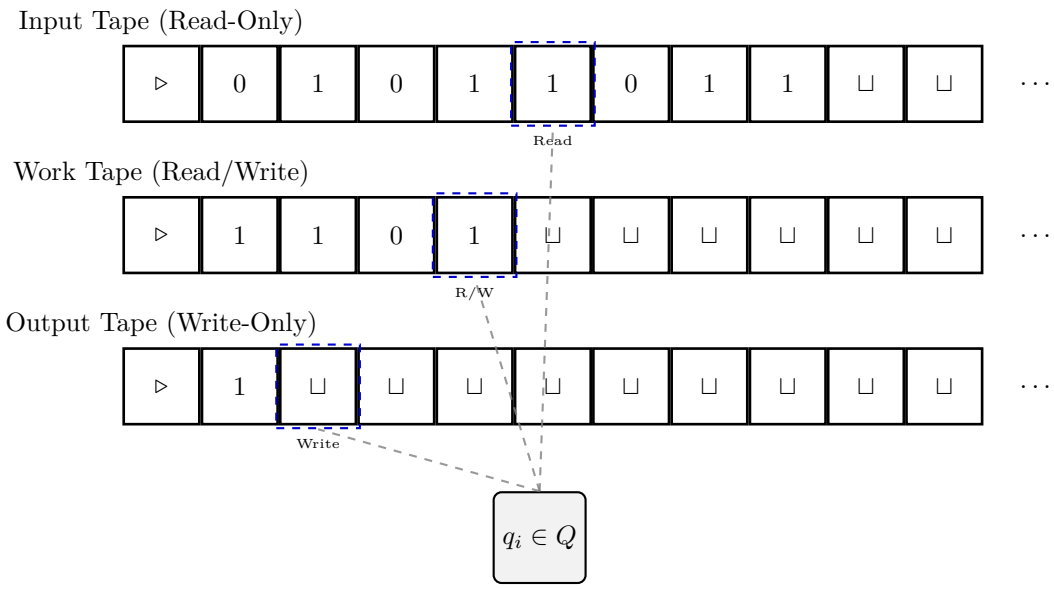

Figure 1: A k-tape Turing Machine (here $k = 3$) with its components.

Definition 2.1 (Deterministic Turing Machine (DTM))**.** A k-tape DTM is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$:

- Q is a finite set of states (the "State Register").
- Σ is a finite input alphabet (not containing the blank symbol $\sqcup$).
- Γ is a finite tape alphabet, where $\Sigma \subset \Gamma$ and $\sqcup \in \Gamma$. It always contains a start symbol $\triangleright \in \Gamma \setminus \Sigma$.
- $\delta : (Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma^k \to Q \times \Gamma^{k-1} \times \{L, S, R\}^k$ is the transition function. (Input tape is read-only, work/output tapes are read/write. S indicates "Stay").
- $q_0 \in Q$ is the start state.
- $q_{\text{accept}} \in Q$ is the accept state.
- $q_{\text{reject}} \in Q$ is the reject state ($q_{\text{accept}} \neq q_{\text{reject}}$).

Definition 2.2 (Configuration)**.** A *configuration* of a k-tape Turing Machine M (as defined in Def. 2.1) is an instantaneous description of its state. Formally, it is a $(k+1)$-tuple $C = (q, (t_1, \ldots, t_k), (h_1, \ldots, h_k))$ where:

- $q \in Q$ is the current state.
- $t_i \in \Gamma^*$ is the finite string on tape i from cell 0 up to the rightmost non-$\sqcup$ symbol.
- $h_i \in \mathbb{N}$ is the head position on tape i (an integer index).

For 1-tape machines: This complex notation is simplified. A configuration can be represented as a single string $uqv \in \Gamma^* Q \Gamma^*$, where:

- uv is the tape content from cell 0 to the rightmost non-$\sqcup$ symbol.
- $q \in Q$ is the current state.

- The head is positioned on the first symbol of v.

We say a configuration C_1 *yields* C_2 in one step, written $C_1 \vdash C_2$, if C_2 is the result of applying the transition function δ to C_1.

Definition 2.3 (Computation and Acceptance)**.** Let M be a k-tape DTM and $w \in \Sigma^*$ be an input string.

- The **start configuration** of M on w is the configuration:

$$C_{\text{start}} = (q_0, (\triangleright w, \triangleright, \ldots, \triangleright), (0, 0, \ldots, 0))$$

  (Here, $\triangleright w$ denotes the string on the input tape, and all other tapes contain only the start symbol $\triangleright$. All heads start at cell 0.)

- M **accepts** w if the start configuration yields a configuration containing q_{accept}.
- M **rejects** w if the start configuration yields a configuration containing q_{reject}.
- M **decides** a language L if it halts on all inputs $w \in \Sigma^*$ (i.e., it always enters either q_{accept} or q_{reject}) and it accepts w if and only if $w \in L$.

Example 2.4 (A DTM for Even Parity)**.**

- $Q = \{q_{\text{even}}, q_{\text{odd}}, q_{\text{accept}}, q_{\text{reject}}\}$ (with $q_0 = q_{\text{even}}$)
- $\Gamma = \{0, 1, \triangleright, \sqcup\}$ (assuming a 1-tape machine)
- δ is defined as follows (for a 1-tape machine, using S for "Stay"):
  - $\delta(q_{\text{even}}, 0) = (q_{\text{even}}, 0, R)$
  - $\delta(q_{\text{even}}, 1) = (q_{\text{odd}}, 1, R)$
  - $\delta(q_{\text{odd}}, 0) = (q_{\text{odd}}, 0, R)$
  - $\delta(q_{\text{odd}}, 1) = (q_{\text{even}}, 1, R)$
  - $\delta(q_{\text{even}}, \sqcup) = (q_{\text{accept}}, \sqcup, S)$
  - $\delta(q_{\text{odd}}, \sqcup) = (q_{\text{reject}}, \sqcup, S)$
  - $\delta(q_{\text{even}}, \triangleright) = (q_{\text{even}}, \triangleright, R)$
  - $\delta(q_{\text{odd}}, \triangleright) = (q_{\text{odd}}, \triangleright, R)$

This machine scans the tape, flipping between q_{even} and q_{odd} when it sees a '1'. It accepts if it is in q_{even} when it hits the end of the tape ($\sqcup$), and rejects otherwise.

Definition 2.5 (Non-deterministic Turing Machine (NTM))**.** An NTM is identical to a DTM, except for its transition function, which allows multiple possible transitions.

- $\delta : (Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma^k \to \mathcal{P}(Q \times \Gamma^{k-1} \times \{L, S, R\}^k)$

Here, $\mathcal{P}(\cdot)$ denotes the power set. The machine may have multiple valid next moves from a given state. An NTM *accepts* an input w if there exists *at least one* sequence of choices (a computation path) that leads to q_{accept}. This creates a "computation tree," as shown in Figure 2.

Example 2.6 (An NTM for Composites)**.**

1. On input x (representing n), non-deterministically "guess" two integers $p, q > 1$ onto a work tape, such that $|p| < |x|$ and $|q| < |x|$.
2. Deterministically (as a subroutine) compute the product $p \times q$.
3. If $p \times q = n$, enter q_{accept}. Otherwise, enter q_{reject}.

If n is composite, there exists a path (a guess of p, q) that leads to acceptance. If n is prime, no such path exists, and all paths will fail. This NTM runs in polynomial time because the certificate (p, q) is shorter than n, and multiplication is a polynomial-time operation.

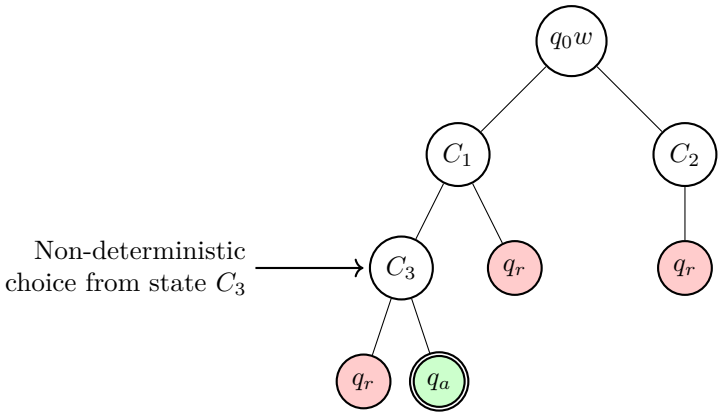

Figure 2: An NTM's computation can be viewed as a tree of possible configurations. The machine accepts if *any* branch reaches an accepting state q_a.

Definition 2.7 (Time Complexity Classes (Barak, Def. 1.12 & 2.5))**.** Let $T : \mathbb{N} \to \mathbb{N}$ be a function.

- **DTIME($T(n)$):** A language L is in DTIME($T(n)$) iff there is a *deterministic* TM M that decides L and runs in time $c \cdot T(n)$ for some constant $c > 0$.
- **NTIME($T(n)$):** A language L is in NTIME($T(n)$) iff there is a *non-deterministic* TM N that decides L and runs in time $c \cdot T(n)$ for some constant $c > 0$. (This means N halts on all branches in $c \cdot T(n)$ steps).

Remark 2.8 (Robustness of the Model)**.** (Barak, Sec 1.3.1, 1.6.1) The class of problems solvable in polynomial time is remarkably robust. It is known that:

- Any k-tape DTM running in $T(n)$ time can be simulated by a single-tape DTM in $O(T(n)^2)$ time.
- Any DTM running in $T(n)$ time can be simulated by an oblivious DTM (where head movements depend only on n, not tape content) in $O(T(n) \log T(n))$ time.
- A Random Access Machine (RAM) model (which is closer to a real PC) can be simulated by a DTM with only a polynomial slowdown (and vice-versa).

This gives us confidence that the class P is not an artifact of the TM model, but a fundamental class of problems.

3 Complexity Classes and Reductions

This section rigorously defines the complexity classes P and NP, drawing from the definitions provided in Barak (2016).

3.1 The Class P

Definition 3.1 (The class P (Barak, Def. 1.13))**.**

$$\mathrm{P} = \bigcup_{c \geq 1} \mathrm{DTIME}(n^c)$$

Example 3.2 (Problems in P)**.**

- **PATH (Graph Connectivity):** (Barak, Ex. 1.14) Given a graph $G = (V, E)$ and two vertices $s, t \in V$, is there a path from s to t? This is decidable in polynomial time (e.g., $O(|V| + |E|)$) using Breadth-First Search (BFS) or Depth-First Search (DFS).
- **PRIMES:** Given an integer n, is n a prime number? This was famously proven to be in P by Agrawal, Kayal, and Saxena in 2002 (the "AKS primality test").

3.2 The Class NP

Definition 3.3 (The class NP (Barak, Def. 2.1))**.** A language $L \subseteq \{0,1\}^*$ is in NP if there exists a polynomial $p : \mathbb{N} \to \mathbb{N}$ and a polynomial-time TM M (called the *verifier* for L) such that for every $x \in \{0,1\}^*$,

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)} \text{ s.t. } M(x,u) = 1$$

If $x \in L$ and $u \in \{0,1\}^{p(|x|)}$ satisfy $M(x,u) = 1$, then we call u a *certificate* for x (with respect to L and M).

Example 3.4 (Problems in NP)**.**

- INDSET**:** (Barak, Ex. 2.2) Given a graph $G = (V, E)$ and an integer k, does G have an independent set of size at least k? (An independent set is a subset of vertices $V' \subseteq V$ such that no two vertices in V' are connected by an edge). *Certificate u:* A subset of k vertices V'. *Verifier M:* Checks that $|V'| = k$ and that for all $v_i, v_j \in V'$, $(v_i, v_j) \notin E$. This takes polynomial time.
- **TSP (Traveling Salesperson):** (Barak, Ex. 2.3) Given n nodes, distances $d_{i,j}$, and a budget k, is there a tour (a circuit visiting every node exactly once) of total length at most k? *Certificate u:* A permutation π of the n nodes. *Verifier M:* Checks that π is a valid permutation and that $\sum_{i=1}^{n-1} d_{\pi(i),\pi(i+1)} + d_{\pi(n),\pi(1)} \leq k$. This takes polynomial time.
- **SUBSET-SUM:** (Barak, Ex. 2.3) Given a list of n numbers $A_1, \ldots, A_n$ and a target T, is there a subset $I \subseteq \{1, \ldots, n\}$ such that $\sum_{i \in I} A_i = T$? *Certificate u:* The subset I. *Verifier M:* Computes the sum $\sum_{i \in I} A_i$ and checks if it equals T. This takes polynomial time.

Proposition 3.5 (Equivalence of NP Definitions (Barak, Thm. 2.6))**.**

$$\mathrm{NP} = \bigcup_{c \in \mathbb{N}} \mathrm{NTIME}(n^c)$$

Proof. This proof demonstrates the formal equivalence between the "verifier" definition (Def. 3.4) and the "non-deterministic computation" definition (Def. 2.7, Prop. 3.8).

Part 1: $\mathrm{NP} \subseteq \bigcup \mathrm{NTIME}(n^c)$ **(Verifier $\implies$ NTM)**

- **Hypothesis:** Let $L \in \mathrm{NP}$. By Def. 3.4, there exists a polynomial $p_M(|x|)$ and a polynomial-time *deterministic* verifier M (which we can assume has k tapes) such that $x \in L \iff \exists u$ with $|u| \leq p_M(|x|)$ where $M(x,u) = 1$.
- **Goal:** Construct a *non-deterministic* TM N that decides L in polynomial time.
- **Construction of N:** On input x (of length n):
  1. **Guessing Phase:** N has an extra work tape. It non-deterministically writes a string u (the certificate) onto this tape. It does this by having two choices for each step for $p_M(n)$ steps: (1) write '0' and move right, or (2) write '1' and move right. This phase takes $p_M(n)$ steps.
  2. **Checking Phase:** After $p_M(n)$ steps, N's head for the certificate tape returns to the start. N now behaves as a *deterministic* machine. It simulates the verifier M, treating x on its input tape and the guessed string u on its new work tape as M's two inputs.
  3. **Halt:** If the simulation of $M(x,u)$ accepts, N enters q_{accept}. If $M(x,u)$ rejects, N enters q_{reject}.
- **Analysis:**
  - **Correctness:** If $x \in L$, then there *exists* a correct certificate u. The computation path of N that guesses this specific u will lead to q_{accept}. If $x \notin L$, no certificate u exists, so *all* computation paths of N will lead to q_{reject}. This satisfies the NTM acceptance condition.
  - **Time:** The guessing phase takes $p_M(n)$ time. The checking phase simulates M. M is a polynomial-time TM, so it runs in time $\mathrm{poly}(|x| + |u|) = \mathrm{poly}(n + p_M(n))$. Since p_M is a polynomial, a polynomial of a polynomial is still a polynomial. Thus, the total time for N is $p_M(n) + \mathrm{poly}(n + p_M(n))$, which is a polynomial in n.

Part 2: $\bigcup \mathrm{NTIME}(n^c) \subseteq \mathrm{NP}$ **(NTM $\implies$ Verifier)**

- **Hypothesis:** Let $L \in \text{NTIME}(p(n))$ for some polynomial $p(n) = n^c$. By Def. 2.7, there exists an NTM N that decides L in $p(n)$ time.
- **Goal:** Construct a *deterministic* verifier M and a polynomial p_M as required by Def. 3.4.
- **Certificate u:** If $x \in L$, there must exist at least one accepting computation path of N on input x. This path is a sequence of non-deterministic choices. At any step i, N's transition δ may offer d possible moves. We can encode the choice taken at step i with a small string. The certificate u is the concatenation of all $p(n)$ of these choices for one accepting path.
- **Certificate Length $p_M(n)$:** The NTM runs for $p(n)$ steps. The number of choices at each step is a constant d (dependent on M, not x). Thus, the length of u is $O(p(n) \cdot \log d) = O(p(n))$. This is a polynomial in $|x|$.
- **Verifier M:** M is a *deterministic* TM that takes x and u as input.
  1. M simulates the NTM N on input x.
  2. At each step i of the simulation, when N would have a non-deterministic choice, M deterministically reads the i-th "choice" from the certificate u and takes that specific path.
  3. If u is ill-formed (e.g., too short, or specifies an invalid choice) or the simulated path does not reach q_{accept} within $p(n)$ steps, M rejects.
  4. If the path specified by u leads N to q_{accept}, M accepts.
- **Analysis:**
  - **Correctness:** M is a deterministic TM. By construction, $M(x, u) = 1$ if and only if u encodes a valid accepting computation path for N on input x. This is exactly the verifier condition.
  - **Time:** The verifier M is just a DTM simulating a *single* path of an NTM. Simulating one step of N takes polynomial time (a constant time lookup in N's transition table and tape operations). Since M only simulates $p(n)$ steps, its total runtime is polynomial.

□

3.3 Reducibility and NP-Completeness

Definition 3.6 (Reductions and NP-Completeness (Barak, Def. 2.7))**.**

- A language $L \subseteq \{0,1\}^*$ is *polynomial-time Karp reducible* to a language $L' \subseteq \{0,1\}^*$ (denoted $L \leq_p L'$) if there is a polynomial-time computable function $f : \{0,1\}^* \to \{0,1\}^*$ such that for every $x \in \{0,1\}^*$,
$$x \in L \iff f(x) \in L'$$
- We say that L' is NP-hard if $L \leq_p L'$ for every $L \in \text{NP}$.
- We say that L' is NP-complete if L' is NP-hard and $L' \in \text{NP}$.

Remark 3.7 (The Power of Reductions)**.** The reduction f acts as a translator. It maps "yes" instances of L to "yes" instances of L', and "no" instances of L to "no" instances of L'. If we have a "black box" algorithm (an oracle) that solves L', we can solve L as follows:

1. Given input x for L.
2. Compute $f(x)$ in polynomial time.
3. Ask the oracle for L' if $f(x) \in L'$.
4. Return the oracle's answer.

This implies that L is "no harder" than L'. The contrapositive is key: if $L' \in \text{P}$ implies $L \in \text{P}$, then $L \notin \text{P}$ (assuming $\text{P} \neq \text{NP}$) implies $L' \notin \text{P}$. This is how we prove hardness.

Theorem 3.8 ((Barak, Thm. 2.8))**.**

1. *(Transitivity) If $L \leq_p L'$ and $L' \leq_p L''$, then $L \leq_p L''$.*

2. *If language L is* NP*-hard and* $L \in \mathrm{P}$*, then* $\mathrm{P} = \mathrm{NP}$.

3. *If language L is* NP*-complete, then* $L \in \mathrm{P}$ *if and only if* $\mathrm{P} = \mathrm{NP}$.

Proof. (From Barak, Thm. 2.8) For (1), if f_1 is a polynomial-time reduction from L to L' and f_2 is a reduction from L' to L'', then the mapping $x \mapsto f_2(f_1(x))$ is a polynomial-time reduction from L to L''. If f_1 runs in $O(n^{c_1})$ and f_2 runs in $O(n^{c_2})$, the composition runs in $O(n^{c_1}) + O((n^{c_1})^{c_2}) = O(n^{c_1 c_2})$, which is a polynomial in n. Parts (2) and (3) follow directly from the definitions. □

3.4 The Class coNP

Definition 3.9 (The class coNP)**.** A language L is in coNP if its complement $\bar{L} = \Sigma^* \setminus L$ is in NP. Equivalently, $L \in \mathrm{coNP}$ if there is a polynomial-time verifier V such that for all x:

$$x \in L \iff \forall y, |y| \leq p(|x|), V(x, y) = 1$$

This class captures problems for which a "no" instance has a short, verifiable certificate (a counterexample). For example, UNSAT (the problem of determining if a formula is *not* satisfiable) is in coNP.

Remark 3.10 (Remark on coNP problems which are not NP-hard)**.** Let L be an arbitrary problem such that $L \in \mathrm{P}$. If L were also NP-hard, then by definition any problem in NP could be reduced to L via a polynomial time algorithm. But since L has a polynomial time solution, then all problems in NP would have a polynomial time solution (first reduce it to L in polynomial time and then solve L in polynomial time). Thus, under the assumption of $\mathrm{P} \neq \mathrm{NP}$, any problem in P (that is not trivial, i.e. not $\emptyset$ or Σ^*) provides a counterexample.

Note that under the assumption of $\mathrm{P} = \mathrm{NP}$, we have that $\mathrm{coP} = \mathrm{coNP}$ and since $\mathrm{coP} = \mathrm{P}$ we obtain $\mathrm{P} = \mathrm{coNP}$. On the other hand, if $\mathrm{P} = \mathrm{NP}$, then every non-trivial problem in P must also be NP-hard. Indeed, any non-trivial problem in P can be reduced to any other non-trivial problem in P. This gives that for any non-trivial problem $L \in \mathrm{P} = \mathrm{NP}$, any NP problem can be reduced to L. Hence L is NP-hard. Then since $\mathrm{P} = \mathrm{coNP}$ we see that it is impossible to find a non-trivial coNP problem which is not NP-hard.

4 The SAT and 3SAT Problems

The definitions in this section are from your original notes.

Definition 4.1 (Boolean formula)**.**

1. (Base case)
   (a) Any variable $x_i \in V$ is a formula.
   (b) The constants $\top$ (True) and $\bot$ (False) are formulas.
2. (Inductive steps)
   (a) If ϕ is a formula, then $\neg\phi$ is a formula (closure under negation).
   (b) If ϕ and ψ are formulas, then $(\phi \vee \psi)$ is a formula (closure under disjunction).
   (c) If ϕ and ψ are formulas, then $(\phi \wedge \psi)$ is a formula (closure under conjunction).

Remark 4.2 (Minimality of the definition)**.** The above definition is minimal in the sense that $\{\neg, \vee, \wedge\}$ is a functionally complete set of logical connectives, meaning that any other logical connective (such as $\rightarrow$ or $\leftrightarrow$) can be built using only this set.

Definition 4.3 (Literal, clause, CNF)**.**

- A *literal* is either a variable x_i (a positive literal) or its negation $\neg x_i$ (a negative literal).
- A *clause* is a disjunction of one or more literals.
  Example: $(x_1 \vee \neg x_2 \vee x_3)$ is a clause.
- ϕ is in *Conjunctive Normal Form (CNF)* if it is a conjunction of one or more clauses.
  Example: $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3)$ is in CNF.

- A formula is in *k-CNF* if it is in CNF and every clause has exactly k literals.

Definition 4.4 (Value of a Boolean formula)**.** If ϕ is a Boolean formula over the variables $V = \{x_1, x_2, \ldots, x_n\}$ and $z \in \{0,1\}^n$. Then we denote by $\phi(z)$ the value of the expression ϕ (where 1 is associated to $\top$ and 0 is associated to $\bot$) when each variable x_i is replaced by the i-th coordinate of z.

Example 4.5. Let $\phi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$ and $z = (0,0,1)$. Then $\phi(z) = (0 \vee 0) \wedge (\neg 0 \vee 1) = 0 \wedge (1 \vee 1) = 0 \wedge 1 = 0 = \bot$. However, for the assignment $z' = (0,1,1)$, $\phi(z') = (0 \vee 1) \wedge (\neg 0 \vee 1) = 1 \wedge (1 \vee 1) = 1 \wedge 1 = 1 = \top$. Thus, ϕ is satisfiable.

Definition 4.6 (Satisfiability)**.** Let ϕ be a Boolean formula over the variables $V = \{x_1, x_2, \ldots, x_n\}$. We say that ϕ is *satisfiable* if there exists some assignment $z \in \{0,1\}^n$ such that $\phi(z) = \top$. In this case, we say that z *satisfies* ϕ. Otherwise, ϕ is called *unsatisfiable*.

Problem 4.7 (SAT)**.** Given a Boolean formula ϕ in CNF, is ϕ satisfiable?

Problem 4.8 (3SAT)**.** Given a 3-CNF Boolean formula ϕ, is ϕ satisfiable?

5 The Cook-Levin Theorem

This section presents the proof of the Cook-Levin Theorem, following the construction from Barak (2016).

Theorem 5.1 (Cook-Levin Theorem (Barak, Thm. 2.10))**.**

1. SAT *is* NP-*complete.*
2. 3SAT *is* NP-*complete.*

Proof (Part 1: SAT $\in$ NP*).* This is required to show NP-complete.

- **Language:** $L = \text{SAT}$.
- **Input:** $x = \phi$, a CNF formula.
- **Certificate** u**:** A truth assignment $z \in \{0,1\}^n$ for the n variables in ϕ. The length of u is n, which is polynomial in the size of ϕ.
- **Verifier** $M(x,u)$**:** A deterministic TM that:
  1. Substitutes the truth values from u into the formula ϕ.
  2. For each clause C_j, checks if at least one literal is $\top$.
  3. If all clauses are satisfied, M outputs 1 (accepts).
  4. If any clause is not satisfied, M outputs 0 (rejects).

  This verifier runs in polynomial time, as evaluating the formula ϕ with a given assignment is efficient.

Since SAT has a polynomial-time verifier, SAT $\in$ NP. □

Lemma 5.2 (SAT is NP-hard (Barak, Lemma 2.11))**.** SAT is NP-hard.

Proof (Part 2: SAT *is* NP-*hard).* Let L be an arbitrary NP language. By definition (Def. 3.4), there is a polynomial-time verifier k-tape TM M and a polynomial p such that for every $x \in \{0,1\}^*$ (let $n = |x|$):

$$x \in L \iff \exists u \in \{0,1\}^{p(n)} \text{ s.t. } M(x,u) = 1$$

We describe a polynomial-time transformation $x \mapsto \phi_x$ from strings to CNF formulae such that ϕ_x is satisfiable if and only if $x \in L$.

Note on the Verifier (1-tape vs k-tape): The definition of NP provides a k-tape verifier M that runs in time $T_k(n)$, which is a polynomial in n. By 'Remark 2.8 (Robustness of the Model)', there exists an equivalent **1-tape DTM**, let's call it M', that also decides the verifier's language. This M' runs in time $T_1(n) = O(T_k(n)^2)$. Since a polynomial squared is still a polynomial, M' is also a polynomial-time verifier.

To simplify the proof, we will construct a formula ϕ_x that simulates this equivalent *1-tape machine* M'. This is valid because M' exists for any $L \in$ NP.

Setup for 1-Tape Simulation:

- **Input x:** The original problem, with length $n = |x|$.
- **Certificate u:** The unknown certificate, with length bounded by $p(n)$.
- **Input to M':** The 1-tape machine M' is given a single tape containing $w' = x\#u$, where # is a special separator symbol.
- **Runtime $T(n)$:** M' runs in time $T(n) = O((n + p(n))^c)$ for some constant c, which is a polynomial in n.
- **Space $S(n)$:** A 1-tape machine running for $T(n)$ steps can use at most $T(n) + 1$ tape cells. Thus, its space is $S(n) = O(T(n))$, which is also a polynomial in n.

The Computation Tableau

The core of the proof is to represent the entire computation history of $M'(x\#u)$ as a static tableau (a grid). The tableau will have $S(n)$ columns (one for each tape cell) and $T(n)$ rows (one for each time step). A **configuration** (Def. 2.2) is a full snapshot of the machine.

The Variables: The formula ϕ_x must be able to describe this $T(n) \times S(n)$ tableau. Its variables are:

- $u_1, \ldots, u_{p(n)}$: The bits of the unknown certificate u. (These are *not* part of the tableau, but are "master" variables that set the tableau's initial state).
- $T_{i,j,s}$: A variable that is $\top \iff$ "at time i, tape cell j contains symbol $s \in \Gamma$".
- $H_{i,j}$: A variable that is $\top \iff$ "at time i, the single head is at tape cell j".
- $Q_{i,q}$: A variable that is $\top \iff$ "at time i, the machine is in state $q \in Q$".

The total number of variables is polynomial in n (this is proven in the Analysis section below).

The Formula ϕ_x: The final formula is $\phi_x = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}}$. A satisfying assignment for ϕ_x *is* a valid, accepting computation history.

1. **ϕ_{cell}:** "Every cell in the tableau is well-defined." For each time $i \in [0, T(n)]$ and tape cell $j \in [0, S(n)]$, exactly one symbol $s \in \Gamma$ can be present. At each time i, the machine is in exactly one state and the head is in exactly one position.

$$\phi_{\text{cell_tape}} = \bigwedge_{i=0}^{T(n)} \bigwedge_{j=0}^{S(n)} \left[\left(\bigvee_{s \in \Gamma} T_{i,j,s} \right) \wedge \left(\bigwedge_{s \neq t \in \Gamma} (\neg T_{i,j,s} \vee \neg T_{i,j,t}) \right) \right]$$

$$\phi_{\text{cell_head}} = \bigwedge_{i=0}^{T(n)} \left[\left(\bigvee_{j=0}^{S(n)} H_{i,j} \right) \wedge \left(\bigwedge_{j \neq k} (\neg H_{i,j} \vee \neg H_{i,k}) \right) \right]$$

$$\phi_{\text{cell_state}} = \bigwedge_{i=0}^{T(n)} \left[\left(\bigvee_{q \in Q} Q_{i,q} \right) \wedge \left(\bigwedge_{q \neq r \in Q} (\neg Q_{i,q} \vee \neg Q_{i,r}) \right) \right]$$

$$\phi_{\text{cell}} = \phi_{\text{cell_tape}} \wedge \phi_{\text{cell_head}} \wedge \phi_{\text{cell_state}}$$

   This is a CNF formula of polynomial size.

2. **ϕ_{start}:** "The first row ($i = 0$) is the start configuration." This formula asserts that at $i = 0$, the 1-tape machine M' is in its start configuration with input $w' = x\#u$. This formula is a conjunction of the following:

State: Machine is in q_0:

$$\phi_{\text{state}} = Q_{0,q_0} \wedge \left(\bigwedge_{q \in Q, q \neq q_0} \neg Q_{0,q} \right)$$

Head: Head is at cell 0:

$$\phi_{\text{head}} = H_{0,0} \wedge \left(\bigwedge_{j=1}^{S(n)} \neg H_{0,j} \right)$$

Tape Content: The tape must be '$\triangleright x_1 \dots x_n \# u_1 \dots u_{p(n)} \sqcup \dots \sqcup$'. This is enforced by a conjunction of five sub-formulas:

(a) **Start Symbol (Cell 0):**

$$\phi_{\text{tape},1} = T_{0,0,\triangleright}$$

(b) **Input x (Cells 1 to n):** (These x_j are constants, not variables)

$$\phi_{\text{tape},2} = \bigwedge_{j=1}^{n} T_{0,j,x_j}$$

(c) **Separator (Cell $n+1$):**

$$\phi_{\text{tape},3} = T_{0,n+1,\#}$$

(d) **Certificate u (Cells $n+2$ to $n+p(n)+1$):** This part links the tableau to the certificate variables u_k. For each k from 1 to $p(n)$, let $j = n+1+k$. We must enforce that $T_{0,j,1} \iff u_k$ and $T_{0,j,0} \iff \neg u_k$. The CNF formula for this is:

$$\phi_{\text{tape},4} = \bigwedge_{k=1}^{p(n)} \Big[(\neg T_{0,n+1+k,1} \vee u_k) \wedge (T_{0,n+1+k,1} \vee \neg u_k) \wedge (\neg T_{0,n+1+k,0} \vee \neg u_k) \wedge (T_{0,n+1+k,0} \vee u_k) \Big]$$

(e) **Blanks (Cells $n+p(n)+2$ to $S(n)$):**

$$\phi_{\text{tape},5} = \bigwedge_{j=n+p(n)+2}^{S(n)} T_{0,j,\sqcup}$$

The full ϕ_{start} is the conjunction of all these parts.

3. $\phi_{\textbf{accept}}$**:** "The machine accepts." This asserts that q_{accept} is active at *some* time i.

$$\phi_{\text{accept}} = \bigvee_{i=1}^{T(n)} Q_{i,q_{\text{accept}}}$$

This is a single large clause.

4. $\phi_{\textbf{move}}$**:** "The transitions are legal." This asserts that for every time i, the configuration at $i+1$ follows from i according to M''s transition function δ. Computation is *local*: the contents of a cell $T[i+1,j]$ are determined *only* by the 2×3 window of cells $(i,j-1),(i,j),(i,j+1)$ and $(i+1,j-1),(i+1,j),(i+1,j+1)$.

$$\phi_{\text{move}} = \bigwedge_{i=0}^{T(n)-1} \bigwedge_{j=0}^{S(n)} \psi_{i,j}$$

Where $\psi_{i,j}$ is a formula asserting that the 2×3 window centered at (i,j) is "legal" according to δ. Since δ is fixed, $\psi_{i,j}$ is a *constant-size* formula (it depends only on Q and Γ, not n).

Analysis of Formula Size: Let $n = |x|$ be the input size. Let $p(n)$ be the certificate polynomial. The 1-tape simulation runs in time $T(n) = O((n+p(n))^c)$, which is a polynomial in n. The space used is $S(n) = O(T(n))$. Let $C_{\text{const}} = |Q| + |\Gamma|$ be the constant machine size.

Number of Variables: The total count is $O(p(n) + T(n) \cdot S(n) \cdot C_{\text{const}})$. This simplifies to $O(p(n) + T(n)^2)$. Since $T(n)$ and $p(n)$ are polynomials in n, the total number of variables is a polynomial in n.

Size of ϕ_{cell}: For each of $O(T(n) \cdot S(n))$ cells/states/heads, we generate $O(C_{\text{const}}^2)$ clauses. The total size is $O(T(n)^2)$, which is a polynomial in n.

Size of ϕ_{start}:

- ϕ_{state}: $O(|Q|)$ clauses (a constant).

- ϕ_{head}: $O(S(n))$ clauses (a polynomial in n).
- $\phi_{\text{tape},1}$ & $\phi_{\text{tape},3}$: $O(1)$ clauses (a constant).
- $\phi_{\text{tape},2}$: n clauses (a polynomial in n).
- $\phi_{\text{tape},4}$: $O(p(n))$ clauses (a polynomial in n).
- $\phi_{\text{tape},5}$: $O(S(n))$ clauses (a polynomial in n).

The sum $O(S(n) + n + p(n))$ is a polynomial in n.

Size of ϕ_{accept}: This is one clause of $O(T(n))$ literals. The size is $O(T(n))$, which is a polynomial in n.

Size of ϕ_{move}: There are $O(T(n) \cdot S(n))$ windows. The formula $\psi_{i,j}$ for each window is $O(C_{\text{const}}^6)$ (a constant size). The total size is $O(T(n) \cdot S(n)) = O(T(n)^2)$, which is a polynomial in n.

The total size of ϕ_x is the sum of these polynomials, which is itself a polynomial in n. The construction of this formula is also a straightforward, polynomial-time process. Thus, $x \in L \iff \phi_x$ is satisfiable. □

Lemma 5.3 (SAT $\leq_p$ 3SAT (Barak, Lemma 2.14))**.** SAT $\leq_p$ 3SAT.

Proof (from Barak, Sec. 2.3.5). We give a polynomial-time transformation that maps a CNF formula ϕ to a 3-CNF formula ϕ' such that ϕ is satisfiable iff ϕ' is satisfiable. We replace each clause C of ϕ based on its length k.

- **Case 1:** $k = 1$**.** $C = (l_1)$. Introduce new variables y_1, y_2. Replace C with:

  $$(l_1 \vee y_1 \vee y_2) \wedge (l_1 \vee y_1 \vee \neg y_2) \wedge (l_1 \vee \neg y_1 \vee y_2) \wedge (l_1 \vee \neg y_1 \vee \neg y_2)$$

  This is satisfiable iff l_1 is true.

- **Case 2:** $k = 2$**.** $C = (l_1 \vee l_2)$. Introduce new variable y_1. Replace C with:

  $$(l_1 \vee l_2 \vee y_1) \wedge (l_1 \vee l_2 \vee \neg y_1)$$

  This is satisfiable iff $(l_1 \vee l_2)$ is true.

- **Case 3:** $k = 3$**.** $C = (l_1 \vee l_2 \vee l_3)$. Keep this clause as is.

- **Case 4:** $k > 3$**.** $C = (l_1 \vee \cdots \vee l_k)$. Introduce $k - 3$ new variables $z_1, \ldots, z_{k-3}$. Replace C with the conjunction of $k - 2$ new clauses:

  $$(l_1 \vee l_2 \vee z_1) \wedge (\neg z_1 \vee l_3 \vee z_2) \wedge (\neg z_2 \vee l_4 \vee z_3) \wedge \cdots \wedge (\neg z_{k-3} \vee l_{k-1} \vee l_k)$$

  If C is satisfied, some $l_i = \top$. We can set the z variables to satisfy ϕ' (e.g., if $l_3 = \top$, set $z_1 = \top$ and $z_2 = \bot$). If ϕ' is satisfied, but C is not (all $l_i = \bot$), then ϕ' requires $(z_1) \wedge (\neg z_1 \vee z_2) \wedge \cdots \wedge (\neg z_{k-3})$. This forces $z_1 = \top, z_2 = \top, \ldots, z_{k-3} = \top$. But the last clause is $(\neg z_{k-3} \vee l_{k-1} \vee l_k) = (\neg z_{k-3} \vee \bot \vee \bot) = (\neg z_{k-3})$, which is $(\bot)$, a contradiction.

This reduction is polynomial-time. This completes the proof of Theorem 5.1. □

6 Proving NP-complete in Practice

The Cook-Levin theorem gives us a starting point. To prove a new problem L is NP-complete, we no longer need to reduce *all* NP problems to it.

Proposition 6.1 (Strategy for proving NP-complete)**.** To prove a language L is NP-complete, we must:

1. Show that $L \in$ NP.
2. Choose a *known* NP-complete problem L' (e.g., 3SAT).
3. Show that $L' \leq_p L$.

This works because L' is NP-hard (all of NP reduces to it), and by transitivity (Theorem 3.5), L also becomes NP-hard.

Remark 6.2 (Why 3SAT? (Barak, Sec. 2.3.6))**.**

1. **Usefulness:** 3SAT is useful for proving the NP-completeness of other problems. It has a very minimal, regular combinatorial structure, making it easy to use in reductions.
2. **Logical Importance:** Propositional logic has a central role in mathematical logic, which is why Cook and Levin were interested in it in the first place.
3. **Practical Importance:** 3SAT is a simple example of *constraint satisfaction problems (CSPs)*, which are ubiquitous in many fields, including artificial intelligence.

6.1 Example 1: 3SAT to CLIQUE

This is the reduction from your original notes.

Problem 6.3 (CLIQUE)**.** Given a graph $G = (V, E)$ and an integer k, does G contain a k-clique (a subset of k vertices where every vertex is connected to every other vertex in the subset)?

Theorem 6.4. CLIQUE *is* NP*-complete.*

Proof. **Part 1:** CLIQUE $\in$ NP The certificate y is a subset of k vertices $V' \subseteq V$. The verifier $V(G, k, V')$ checks:

1. $|V'| = k$.
2. For every pair of distinct vertices $u, v \in V'$, it checks if the edge $(u, v) \in E$.

This takes $O(k^2)$ edge lookups, which is polynomial in the size of G.

Part 2: CLIQUE **is** NP**-hard** We show 3SAT $\leq_p$ CLIQUE. Given a 3-CNF formula ϕ with m clauses, $C_1, \ldots, C_m$, we must construct a graph G and an integer k in polynomial time.

$$\phi = (l_{1,1} \vee l_{1,2} \vee l_{1,3}) \wedge \cdots \wedge (l_{m,1} \vee l_{m,2} \vee l_{m,3})$$

1. **Construct G:** Create a vertex in G for *every literal* in ϕ. The vertices are grouped into m "clusters" of 3, one for each clause C_j.
2. **Add edges:** Add an edge between two vertices $v_{i,a}$ (literal a in clause i) and $v_{j,b}$ (literal b in clause j) if and only if:
   - $i \neq j$ (they are in different clauses), AND
   - $l_{i,a}$ is not the negation of $l_{j,b}$ (e.g., $l_{i,a} \neq \neg l_{j,b}$). This allows x_1 and x_1, or x_1 and $\neg x_2$, but not x_1 and $\neg x_1$.
3. **Set k:** Set $k = m$ (the number of clauses).

This construction is polynomial time.

Claim: ϕ is satisfiable $\iff$ G has a k-clique.

Proof. ($\Longrightarrow$) Suppose ϕ is satisfied by an assignment z. Then in each clause C_j, at least one literal is $\top$. For each clause C_j, pick *one* such true literal $l_{j,a}$ and select its corresponding vertex $v_{j,a}$. This gives us $k = m$ vertices. Let $v_{i,a}$ and $v_{j,b}$ be two such chosen vertices ($i \neq j$).

- They are in different clauses, so the first edge condition is met.
- Since z makes both $l_{i,a}$ and $l_{j,b}$ true, they cannot be negations of each other (e.g., x_1 and $\neg x_1$). Thus, the second edge condition is met.

An edge exists between every pair. This is a k-clique.

($\Longleftarrow$) Suppose G has a k-clique V'. Since $k = m$ and no edges exist *within* a clause's 3-vertex cluster, the clique V' must contain exactly one vertex from each of the m clusters. We can create a truth assignment z from V':

- For each x_i, if a vertex $v_{j,a}$ corresponding to x_i is in V', set $x_i = \top$.
- If a vertex $v_{j,b}$ corresponding to $\neg x_i$ is in V', set $x_i = \bot$.

This assignment is consistent. Why? If it were not, we would have set $x_i = \top$ (from $v_{j,a}$) and $x_i = \bot$ (from $v_{p,q}$). This means $v_{j,a}$ corresponds to x_i and $v_{p,q}$ corresponds to $\neg x_i$. But by construction, there is no edge between such vertices, so they could not both be in the clique V'. This consistent assignment z satisfies ϕ, because by selecting one vertex from each clause, we have selected one literal from each clause to be $\top$. □

□

6.2 Example 2: 3SAT to INDSET

This is another classic reduction.

Problem 6.5 (INDSET)**.** Given a graph $G = (V, E)$ and an integer k, does G contain an independent set of size at least k?

Theorem 6.6. INDSET *is* NP*-complete.*

Proof. **Part 1:** INDSET $\in$ NP As shown in Example 3.6, the certificate is a subset of k vertices V'. The verifier checks in polynomial time that no two vertices in V' are connected by an edge.

Part 2: INDSET **is** NP**-hard** We show 3SAT $\leq_p$ INDSET. This reduction is remarkably similar to the one for CLIQUE. Given a 3-CNF formula ϕ with m clauses, $C_1, \ldots, C_m$:

$$\phi = (l_{1,1} \vee l_{1,2} \vee l_{1,3}) \wedge \cdots \wedge (l_{m,1} \vee l_{m,2} \vee l_{m,3})$$

1. **Construct G:** Create a vertex in G for *every literal* in ϕ.
2. **Add edges:** Add an edge between two vertices $v_{i,a}$ and $v_{j,b}$ if:
   - $i = j$ (they are in the *same* clause). This forms a triangle (a 3-clique) for each clause.
   - $i \neq j$ AND $l_{i,a} = \neg l_{j,b}$ (they are in different clauses but are contradictory literals).
3. **Set k:** Set $k = m$ (the number of clauses).

This construction is polynomial time.

Claim: ϕ is satisfiable $\iff$ G has an independent set of size k.

Proof. ($\Longrightarrow$) Suppose ϕ is satisfied by an assignment z. Then in each clause C_j, at least one literal is $\top$. For each clause C_j, pick *one* such true literal $l_{j,a}$ and select its corresponding vertex $v_{j,a}$. This gives us a set V' of $k = m$ vertices. Let $v_{i,a}$ and $v_{j,b}$ be two distinct vertices in V'.

- Can an edge exist between them?
- **Case 1:** $i = j$. This is impossible. We only picked *one* vertex from each clause cluster.
- **Case 2:** $i \neq j$. An edge exists only if $l_{i,a} = \neg l_{j,b}$. But this is impossible, as z makes both $l_{i,a}$ and $l_{j,b}$ true, and an assignment cannot make a literal and its negation both true.

Since no edge exists between any two vertices in V', it is an independent set of size k.

($\Longleftarrow$) Suppose G has an independent set V' of size $k = m$.

- Because all vertices within a clause-cluster are connected (as a triangle), V' can contain at most one vertex from each cluster.
- Since $|V'| = k = m$, V' must contain *exactly one* vertex from each of the m clauses.

We can create a truth assignment z from V' just as we did for CLIQUE. This assignment is consistent, because if V' contained $v_{i,a}$ (with literal x_p) and $v_{j,b}$ (with literal $\neg x_p$), an edge would exist between them, contradicting that V' is an independent set. This consistent assignment z satisfies ϕ, because by selecting one vertex from each clause, we have selected one literal from each clause to be $\top$ (based on our assignment). □

□